The Syntax Surgeon

Abstract

The Syntax Surgeon is an intelligent AI-driven assistant designed to provide high-quality conversational interactions, automated Python code analysis, syntax correction, formatting, and documentation export—all without relying on external API keys. By integrating the IBM Granite 3.3 2B Instruct model with advanced Python tooling (AST, regex detection, Black, autopep8) and a modern Gradio interface, the platform provides an accessible and production-ready solution for both conversational AI and technical programming assistance.

1. Introduction

As AI becomes central to software engineering workflows, developers require tools that can not only converse naturally but also analyze, fix, and format code automatically. Many AI tools today rely on cloud APIs, incur ongoing costs, or pose data privacy concerns.

The Syntax Surgeon addresses these challenges by offering:

- Local model execution
- Python code diagnosis and correction
- A powerful conversational interface
- Automatic export options (PDF/TXT)
- A user-friendly Gradio interface

This document provides an in-depth overview of the architecture, workflows, implementation details, and use cases of The Syntax Surgeon.

2. Key Features & Capabilities

Conversational AI Integration

- Built with IBM Granite 3.3 2B Instruct
- Multi-turn conversation handling
- Context-aware responses

Python Code Assistance

- Automatic code detection via regex
- Syntax validation using AST
- Intelligent code fixing
- Automatic formatting (Black, autopep8)

Interaction Management

- Persistent conversation history
- Reset capability
- Text & PDF export
- Shareable conversation links (Pastebin)

Deployment

- Fully operable in Google Colab
- Supports GPU acceleration
- No external API keys required

3. Practical Scenarios

Scenario 1: Python Developer Code Assistance

Automatically fixes:

- Syntax errors
- Missing parentheses/brackets
- Indentation issues
- Formatting problems

Scenario 2: Learning & Tutoring Platform

Ideal for programming educators:

- Explains code line-by-line
- Detects common student errors
- Provides structured feedback

Scenario 3: Code Review & Quality Assurance

QA teams use it to:

- Scan pull request code
- Recommend improvements
- Standardize formatting

Scenario 4: Internal Code Formatting Service

Organizations benefit from:

- Local model deployment
- Zero data leaks
- Reduced API costs

Scenario 5: Interactive Pair Programming

Teams collaborate through:

- Real-time suggestions
- Conversation-based documentation
- Exportable discussion logs

4. Architecture Overview

The Syntax Surgeon consists of four major subsystems:

1. Model Layer – IBM Granite LLM
2. Python Code Analysis Layer – AST, regex, formatters
3. Conversation Layer – message storage & context
4. User Interface Layer – Gradio application

Core Technologies

- Python 3.8+
- IBM Granite 3.3 2B Instruct
- Gradio 4.x
- Transformers (Hugging Face)
- PyTorch
- Black & autopep8
- ReportLab
- Regex

This modular stack ensures reliability, flexibility, and powerful real-time interaction.

5. Software Requirements

- Python 3.8 or above
- pip
- Google Colab
- Gradio, Transformers, Torch
- Black & autopep8 for formatting
- ReportLab for PDF exports

Hardware Requirements

- CPU (minimum)
- GPU recommended: NVIDIA T4, P100, V100
- RAM: 8GB minimum, 16GB preferred
- Storage: 2GB

Knowledge Prerequisites

- Python fundamentals
- NLP basics
- Understanding of AST & code formatting
- Familiarity with model inference frameworks

6. Project Workflow

The development pipeline contains five major milestones:

1. Environment Setup
2. Python Code Analysis Engine
3. AI Response Generator
4. Gradio Interface
5. Testing & Deployment

Each of these is expanded in later sections.

7. Milestone 1: Model Configuration

Activity 1.1 – Install Dependencies

Create requirements file and install:

- gradio
- transformers
- torch
- black
- autopep8
- reportlab
- requests

Activity 1.2 – Chatbot Configuration

The ChatbotConfig class defines:

- Model name
- Temperature
- Top-p
- Max tokens
- Device auto-detection

This ensures controlled generation quality.

8. Milestone 2: Code Analysis Engine

Activity 2.1 – detect_python_code()

Regex patterns identify:

- Functions (def)
- Classes (class)
- Imports
- Control flow (if, for, while)

The system robustly detects code, even when pasted with errors.

Activity 2.2 – check_syntax()

AST parsing provides:

- Accurate syntax detection
- Line-number error reporting
- Detailed error messages

This step ensures precise error localization.

Activity 2.3 – Code Fixing Engine

1. fix_code_indentation()

Handles:

- Tabs vs spaces
- Trailing whitespace
- Misaligned blocks

2. fix_code_brackets()

Automatically closes:

- Parentheses
- Brackets
- Braces

3. format_code()

Primary: Black
Fallback: autopep8
Ensures PEP 8 compliance and clean output.

9. Milestone 3: Conversational AI System

Activity 3.1 – Conversation History

Stores conversation using:

- Role: user/assistant
- Content: message text

Features include:

- Reset
- Export
- Full session context

Activity 3.2 – Response Generation

generate_response() performs:

- Append user message
- Generate model output
- Append assistant response
- Handle token limits
- Apply temperature & top-p

The result is a natural, coherent conversational flow.

10. Milestone 4: Gradio Interface

Activity 4.1 – Chat Interface

Includes:

- Chat window
- Text input
- Send button
- Parameter sliders
- Scrollable history

Activity 4.2 – Export Features

TXT Export

- Clean readable text
- Timestamps
- Easy sharing

PDF Export (ReportLab)

- Color-coded messages
- Paginated
- Professional layout

These exports enhance documentation and collaboration.

Activity 4.3 – Sharing & Collaboration

Pastebin integration provides:

- Public/private links
- Syntax highlighting
- Chat sharing for peer review

11. Milestone 5: Testing & Deployment

Activity 5.1 – Google Colab Deployment

Includes:

- GPU setup
- Model loading
- Full interface testing
- Public URL for sharing

12. Functional Testing

Test Case 1: General Chat

Query: Explain list comprehensions
Result: Model provides structured explanation

Test Case 2: Code Repair

Input: broken Python code
Output: corrected, formatted code

13. Future Enhancements

Potential expansions:

- Support for more languages (JS, C++, Java)
- IDE plug-ins
- Advanced linting (pylint, flake8)
- Real-time streaming responses
- Voice input/output
- Code visualization tools
- Auto-debugging suggestions
- Model fine-tuning options

These enhancements would evolve the platform into a holistic developer assistant.

14. Conclusion

The Syntax Surgeon demonstrates how a local, privacy-safe AI system can combine advanced conversational abilities with intelligent Python code analysis and fixing. By integrating IBM’s Granite model with powerful Python tooling and a polished Gradio interface, it delivers a reliable, extensible, and cost-effective solution for developers, educators, and teams.

The platform is modular, scalable, and adaptable to future enhancements—positioning it as a next-generation AI assistant for modern development workflows.